SABCS: Sparse Attention-Based Code Summarization

Qing Mi[†]
Beijing University of Technology
Beijing, China
miqing@bjut.edu.cn

Yufan Han[*]
Beijing University of Technology
Beijing, China
hyf@emails.bjut.edu.cn

Guoxiang Jiang[*]
Beijing University of Technology
Beijing, China
jiangguoxiang@emails.bjut.edu.cn

Abstract

Code Summarization refers to automatically generating natural language descriptions to explain a piece of code. Existing research primarily relies on traditional transformer models based on self-attention mechanisms. However, this approach encounters issues such as attention redundancy and high computational resource consumption when dealing with long code snippets. To solve this problem, in this paper, we propose a model named SABCS for handling summary generation of long code snippets. This approach employs *Global-local* and *Top-k* attention mechanisms and introduces a multi-dimensional structure-aware attention mechanism based on *AST* and *CFG*. By designing attention rules that guide the focus towards structural information of the code, we simplify the attention matrix and improve the quality of long code summaries. Evaluation experiments conducted on a subset of the publicly available CodeXGLUE benchmark and the dataset contributed by Hu et al., consisting of code snippets with more than 1024 tokens. The results demonstrate that the model significantly outperforms previous models across three evaluation metrics: BLEU-4, METEOR, and ROUGE-L. In summary, our model effectively captures long-distance contextual information and comprehends the semantic and syntactic structures of code, resulting in better summaries for long code snippets.

CCS Concepts: • **Software and its engineering → Software creation and management**.

Keywords: Code Summarization, Sparse Attention Mechanism, Abstract Syntax Tree, Control Flow Graph

ACM Reference Format:
Qing Mi[2], Yufan Han[1], and Guoxiang Jiang[1]. 2024. SABCS: Sparse Attention-Based Code Summarization. In *Woodstock '18: ACM Symposium on Neural Gaze Detection, June 03–05, 2018, Woodstock, NY*. ACM, New York, NY, USA, 10 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/24/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Code Summarization is an essential task that uses computer techniques to automatically generate high-quality, easy-to-understand natural language descriptions. Its primary objective is to extract the core functionality of code implementations, aiding developers in quickly comprehending the content of the source code. As software projects become increasingly complex, there is an increasing demand in the field of software engineering for rapid and precise code comprehension[13, 20].

Early research predominantly utilized methods based on Information Retrieval Techniques[10]. Despite making advancements, it faced challenges such as resource dependency, limited understanding of code semantics and syntax, and restrictions in generalization ability.

With the advancements in deep learning and large-scale language models, Transformer-based models have shown promising outcomes in code summarization[23]. However, the memory and computational complexity of self-attention mechanism in Transformer grows quadratically with the length of the sequence[4]. Consequently, code length is often limited to 512 tokens, which it is insufficient for handling long code typically found in industrial scenarios.

To address this challenge, researchers have been exploring various methods. Currently, one of the mainstream approaches is to enhance the representation of code features during pre-training[6, 7, 14, 19, 22], aiming to improve the ability to capture semantic information. However, despite the superiority of these methods over traditional feature engineering in perceiving code structures, we found that they are limited in handling long code with complex logical structures. The incorporation of code representation methods leads to a significant increase in training input size, resulting in attention redundancy and requiring substantial computational resources.

Therefore, we propose a novel method called Sparse Attention-Based Code Summarization (SABCS), which involves modifying the attention mechanism within the Transformer framework to learn to focus on the structural information from various code representations. In SABCS, we first employ

[*]Both authors contributed equally to this research.
[†]Corresponding authors.

a Global-local attention mechanism to capture long-range dependencies and contextual associations within local regions in the sequence. Following this, we introduce Top-k attention mechanism to select the top k tokens with the highest attention scores to focus on. Subsequently, we establish a multi-dimensional structure-aware attention mechanism based on Abstract Syntax Tree (AST)[15] and Control Flow Graph (CFG)[1]. We set two attention rules to enhance the model's understanding of deeper semantic and syntactic aspects of the code. These mechanisms enable the model to focus on key information from the code perspective while effectively reducing computational overhead, thus mitigating the attention redundancy and computational waste that fully connected attention mechanisms may cause.

Overall, our contributions are as follows:

- We have developed a sparse attention mechanism guided by code structure, introducing Global-local and Top-k attention mechanisms, and designed structure-aware attention mechanisms based on Logic Graph.
- We combined CFG and AST to construct Logic Graph, enabling the model to effectively represent syntactic and semantic-level structural information of code.
- We proposed two attention rules based on Logic Graph. Rule 1 aims to capture long-distance dependency information in code, while Rule 2 lets the model concentrate on tokens with abundant structural details.
- On the [??] dataset[11], our model achieves an average BLEU score of ?.??, which represents a ?% improvement over the previous best result of ?.?? obtained by [baseline model]. Through ablation studies, we find that the key contribution comes from Rule 1. Without this component, the performance drops by A.

2 Related Work

2.1 Code Summarization

In the field of code summarization research, traditional methods mainly rely on extractive approaches based on information retrieval techniques[10]. These methods typically depend on manually designed features and rules to extract key information from code snippets and generate corresponding summaries. However, when identifiers and method names are poorly named, the extracted keywords may be meaningless words or abbreviations, leading to poor quality summaries generated by these methods.

To overcome the limitations of traditional methods, researchers have begun exploring the application of deep learning-based abstractive methods for code summarization tasks. Models such as CodeBERT[6] proposed by Feng et al. have made significant progress in the code summarization field by employing multimodal pre-training techniques. However, when dealing with longer code sequences, these models face challenges such as high computational resource demands and maximum sequence length limitations. To address this issue, Liu et al. introduced the SASA[14] model, which combines structure-aware sparse attention mechanisms to improve performance. Similarly, Gao et al.'s SG-Trans[7] model integrates code structure information into the Transformer architecture to better capture the hierarchical features of code structures. Furthermore, Sun et al. proposed an extract-and-generate framework, called EACS[19], for generating code summaries. It performs the tasks of extracting code outlines and generating summaries through extract and generate modules, preserving practical details similar to human-written summaries.

In summary, although these models have made significant advancements in the field of code summarization, they still face limitations and challenges. Most deep learning methods treat code as plain text, overlooking many code-specific structural details. Some researchers have started to explore richer structural representations of code; however, this often involves increasing the number of embedding neurons and may not be suitable for all long code summarization tasks.

2.2 Spare Attention Models

Sparse attention models are a class of models that modify the attention mechanism within the Transformer framework. In contrast to traditional global attention, sparse attention models only select a small subset of positions relevant to the current position for computation. This strategy effectively reduces computational complexity, making it particularly suitable for handling long sequences or large inputs. By more efficiently allocating attention, these models can maintain focus on key information while lowering computational costs.

In recent years, with the rapid advancement of deep learning technology, researchers have proposed innovative methods to improve sparse attention mechanisms, demonstrating remarkable performance across various domains.

Guo et al.'s Star-Transformer[9] introduces a sparse connected star-shaped topology, effectively reducing the model's reliance on training data. Choromanski et al.'s Performer[5] model utilizes the FAVOR+ method to achieve efficient softmax attention with linear time and space complexity. Wang et al.'s Linformer[21] model employs low-rank matrix approximation, reducing the complexity of self-attention from $O(n^2)$ to O(n), thus saving memory and computational resources. Beltagy et al.'s Longformer[3] model addresses the performance issues of Transformer models when dealing with long documents by introducing linear attention mechanisms. Additionally, Zaheer et al.'s BigBird[24] model has made significant advancements in handling local and global dependencies in long sequences by introducing window and global attention mechanisms, as well as the concept of block-sparse attention, reducing computational complexity while maintaining high performance.

Despite the significant progress in improving computational efficiency and interpretability achieved by these sparse

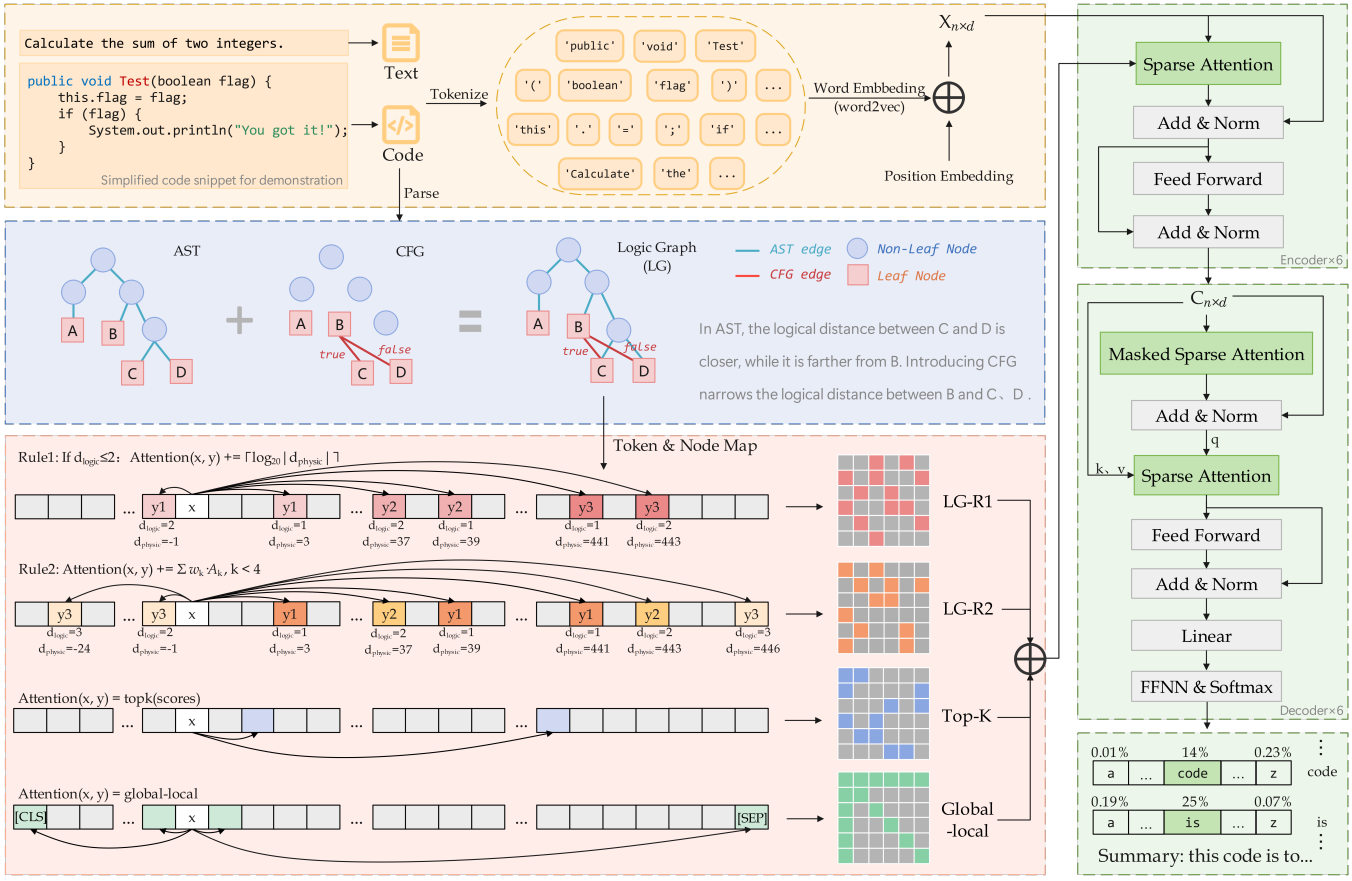


Figure 1. Overall Architecture of SABCS

attention models, they also have some drawbacks. In particular, their innovations primarily focus on improving the attention mechanism itself, overlooking the extraction and utilization of code multi-dimensional features. To address this issue, our research direction is to use a code structure-guided sparse attention mechanism to effectively represent the syntax and semantics of code while reducing computational costs by focusing on a few key positions.

3 SABCS

In this section, we provide a detailed overview of our proposed Sparse Attention-Based Code Summarization (SABCS) model. A schematic overview of this framework is presented in Figure 1. By introducing two appropriate rules, we can adaptively focus on the logical graph(LG) synthesized from fine-grained Abstract Syntax Trees (AST) and Control Flow Graphs (CFG). This allows the model to better capture structural features and long-term dependencies in the code, effectively concentrating attention on the most critical information within long code sequences. In this section, we will introduce our model from three perspectives: an overall overview, the construction of the logical graph, and the implementation of the sparse attention mechanism based on the logical graph.

3.1 Overview

SABCS aims to generate readable code summaries for long code sequences, which can be represented as a supervised natural language generation task. Given source code $C = \{c_1, c_2, \ldots, c_n\}$ and corresponding code summaries $S = \{s_1, s_2, \ldots, s_n\}$, the model learns the mapping relationship between long codes and summaries through a large number of training samples with correct answers, finding a series of parameters θ that maximize the probability of the model generating the code summary $\hat{s}$ for a given input of source code C during testing data. Its theoretical description is as follows.

$$\underset{\theta}{argmax} \sum_{i=1}^{m} \log p_{\text{model}} \left(\hat{s}^{(i)} \mid c^{(i)}; \theta \right) \tag{1}$$

Within this context, m represents the number of samples in the dataset, while θ denotes a series of parameters within the model. Based on the Transformer architecture, we designed three components for SABCS: the encoder, decoder, and sparse attention mechanism based on Logic Graph(LG).

During the data preprocessing stage, we captured features from both the sequential and structural aspects of the source code. To represent sequential information, we utilized the word embedding model Word2Vec[16] to convert each word or token in code snippets and summaries into dense, low-dimensional vector representations. These word embedding vectors were then inputted into the encoder layers. Additionally, we parsed source code snippets using Tree-sitter and PROGEX to obtain Abstract Syntax Trees (ASTs) and Control Flow Graphs (CFGs) corresponding to the code snippets. Based on a hybrid representation of AST and CFG, we constructed a logical graph to extract information related to the code structure, as illustrated in Figure 2.

After establishing the mapping between nodes and code tokens, we devised two attention rules based on the logical graph, as described in Section 3.3. The first rule aims to capture and focus on tokens with longer physical distances but shorter logical distances. Its purpose is to better capture long-distance dependency information in lengthy code, thereby overcoming limitations imposed by code length. However, rule 1 tends to cause attention to be excessively distributed to long-distance tokens. Therefore, we introduced rule 2. Rule 2 calculates the logical distance between tokens and allocates attention based on the strength of their logical relationships. It effectively mitigates the problem of excessive attention caused by rule 1.

Subsequently, we apply these rules to the mapping results, prioritizing token pairs with shorter logical distances and longer physical distances, and adaptively allocate attention to nodes with shorter logical distances.

Following this, we constructed a pattern of attention based on the logical graph (LG) according to these rules. Additionally, we employed the Global-local attention and Top-k attention mechanisms proposed in the field of pre-training for long sequences of text to deal with long code summary generation tasks.

Unlike some models that use graph neural networks or reinforcement learning to form multiple encoders (such as Cast[18], GypSum[22], GraphCodebert[8]), our model only takes original code and summaries as training inputs, mapping code snippets to a fixed-length context vector containing relevant semantic information. The encoder consists of 6 Transformer encoder layers, each comprising two sub-layers: the sparse attention mechanism we propose and feed-forward neural network layers. These sub-layers are connected via residual connections and layer normalization operations.

$$H^n = transformer_n(H^{n-1}), n \in [1, N] \tag{2}$$

The decoder initially receives the context vector output by the encoder as the initial state vector, serving as the starting point for generating summaries. Subsequently, based on the initial state vector and a specific generation strategy based on the sparse attention mechanism, summaries are generated incrementally. Finally, predictions are made within the generated sequence to determine the end marker for the summary.

3.2 Construction of the Logic Graph

In this section, we will discuss the methods for analyzing and representing code structures, and introduce the definition of logical graphs in this paper. Building on this foundation, we will provide a detailed explanation of the concepts of logical distance and physical distance, which are crucial metrics for evaluating the relationships between tokens in code structures. Through a thorough discussion of these concepts, we aim to gain a better understanding of the importance of code structure.

Abstract Syntax Tree The Abstract Syntax Tree (AST)[15] is a tree-like data structure used to represent the syntactic structure of source code in programming languages. It provides a structured representation of the code, capturing important elements such as statements, expressions, variables, and control flow constructs. Each node in the tree represents a specific syntactic construct in the source code, and the relationships between nodes reflect the relationships between these constructs. For example, a binary operation expression may have two child nodes representing operands and a parent node representing the operation itself. As an intermediate representation of the code, AST can be further analyzed, optimized, and transformed by compilers, interpreters, and other language processing tools. It is an essential tool for tasks such as code analysis, optimization, refactoring, and transformation.

Control Flow Graph Control Flow Graph (CFG)[1] is a directed graph structure used to represent the execution paths of program code. In a Control Flow Graph, nodes represent basic blocks in the program, while edges represent control flow transitions, indicating the flow of program execution. Control Flow Graphs aid programmers in understanding the structure and logic of a program and are utilized in software engineering for code analysis, optimization, testing, and other areas. Integrating Control Flow Graphs with Abstract Syntax Trees (AST) can provide programmers with a more comprehensive analysis and understanding of the code, as each captures different aspects of program information.

Logical Graph A logical graph combines abstract syntax trees (AST) and control flow graphs (CFG) into a composite graph structure used to represent the logical and control flow processes of program code. It can be represented as a quadruple:

$$G = (V, E, L, C) \tag{3}$$

Where V is the set of nodes in the graph, each representing a basic statement or expression in the program. E is the set of edges in the graph, each representing control flow or data

https://github.com/tree-sitter/tree-sitter
https://github.com/ghaffarian/progex

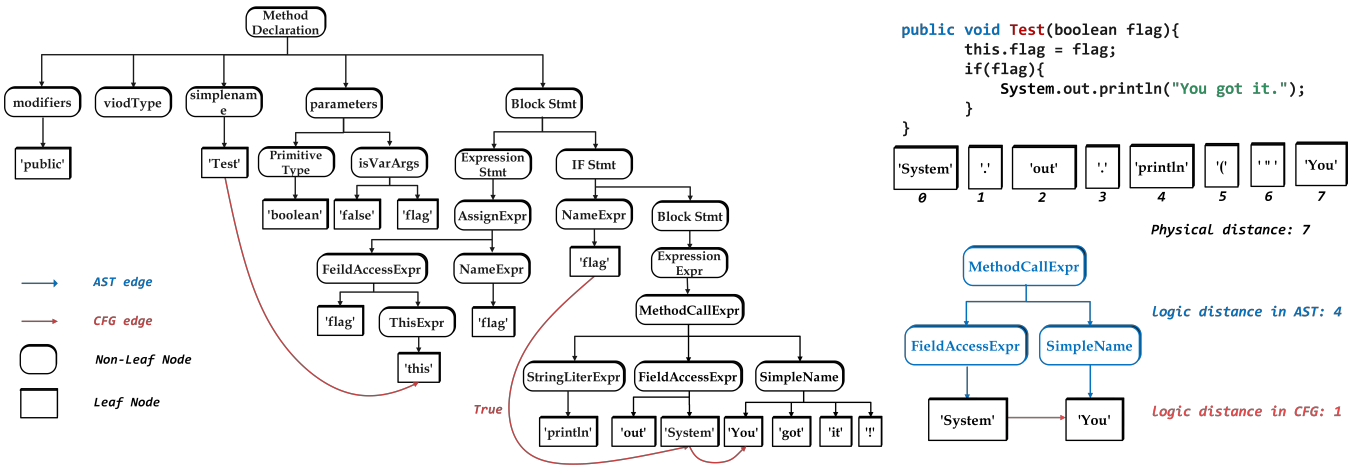


Figure 2. Construction of the logical diagram

flow relationships in the program. L is the mapping function from nodes to labels, recording the mapping relationship between nodes and code tokens. C is the mapping function from edges to conditions, mapping control flow edges to conditions indicating control flow transitions.

Logical Connectivity Let n represent a node in the logical directed graph G, which has two attribute values - node type d and code token t. The path between nodes in G starting from terminal node n_1, passing through a series of nodes $n_2, n_3, \ldots$, and reaching final node n_{k+1} is referred to as logical connectivity between nodes n_1 and n_{k+1}.

Logical Distance $d_{\log}(i, j)$ The logical distance between token pairs (i, j) represents the proximity of token i to token j in code logic, independent of their actual positions in the code sequence but dependent on the positions of node pairs in the logical directed graph. The formula is as follows:

$$\hat{d}_{\log}(i, j) = \text{mindis}(i, j) \tag{4}$$

Where $\text{mindis}(i, j)$ represents the shortest path length between nodes i and j in the logical directed graph.

Physical Distance $d_{\text{phy}}(i, j)$ The physical distance between token pairs (i, j) represents the absolute difference in their actual positions in the code sequence, as shown in the formula:

$$d_{\text{phy}}(i, j) = |\text{location}(i) - \text{location}(j)| \tag{5}$$

Where $\text{location}(i)$ represents the actual position of token i in the code sequence, represented by an integer.

3.3 Sparse Attention Mechanism Based On the LG

In the preceding overview section, we introduced the sparse attention modules, Top-k and Global-local, utilized in our work. In this section, we focus on presenting our novel sparse attention mechanism based on the Logic Graphs (LGs), which is designed based on two specific rules.

Abstract Syntax Trees (ASTs) and Control Flow Graphs (CFGs) provide crucial information about code syntax structure and execution paths. However, using these representations alone may lead to information loss. Therefore, we employ the open-source parsers Tree-sitter and PROGEX to respectively construct ASTs and CFGs for code structure analysis. We then reconstruct logical graphs (LGs) from these representations. In the LG, we establish mappings between tokens and LG nodes, and store node type features and attribute labels in the form of adjacency matrices.

Rule 1 aims to capture and focus on token pairs with large physical distances but small logical distances. Specifically, for token pairs with a physical distance exceeding a but a logical distance less than or equal to n, the corresponding value is incremented by 1 based on the original value. The design of this rule aims to focus on long-distance dependency information in the code, thereby enhancing understanding and analytical capabilities of the code structure.

The definition of this rule is as follows:

$$M_{ij} = \begin{cases} \lceil \log_a |d_{\text{phy}}(i, j)| \rceil, & \text{if } d_{\log}(i, j) \leq n \\ -\infty, & \text{otherwise} \end{cases} \tag{6}$$

Here, $d_{\text{phy}}(i, j)$ represents the physical distance between token pairs, $d_{\log}(i, j)$ represents the logical distance between token pairs, and n is the threshold for logical distance. Since code typically has many nodes and edges in the logical graph, we set n to 2. Through multiple experimental tests, we determined the optimal value for a to be 20.

Rule 2 aims to fine-tune the distribution of node attention by analyzing the logical strength between tokens, assigning different levels of attention to nodes at different layers. This approach optimizes the use of distance information between tokens, promoting the model's learning of subtle differences in the code structure. Implementing this rule requires constructing adjacency matrices in different dimensions and their respective weights, as well as establishing mappings

between tokenized tokens and AST nodes. Through this systematic process, attention allocation is refined, prioritizing the proximity between nodes. This refinement enhances the appropriate attention thresholds for complex structural features in the code.

$$M_{ij} = \sum_{k=1}^{n} w_k \cdot A_k \tag{7}$$

Where M_{ij} represents the consolidated weighted adjacency matrix, A_k denotes the adjacency matrix corresponding to the k-th order, and w_k signifies the weight attributed to the k-th adjacency matrix, which needs to be determined through extensive experimentation.

4 Experimental Setup

For our experimental evaluation, we employ Java datasets as benchmarks and utilize BLEU-4, METEOR, and ROUGE-L metrics as assessment criteria. To ensure a fair comparison, we replicate all baselines and conduct experiments using the same dataset.

4.1 Data Set

To create datasets suitable for generating long code summaries, we utilized two existing datasets: JAVH, used by Hu et al., consisting of 78,422 source code-summary pairs, and JAVC, provided by CodeXGLUE, containing 164,923 source code-summary pairs. We selected data from both datasets where the code length exceeds 1024 tokens and excluded data with fewer than two nodes in the Abstract Syntax Trees (ASTs). Following this filtering process, the JAVH dataset comprises 7,749 pairs of source code and summaries, while the JAVC dataset includes 22,547 pairs. We partitioned the JAVH and JAVC datasets into training, validation, and test sets in an 8:1:1 ratio.

The statistical details of the filtered datasets are summarized in Table 1, which presents the number of code-summary pairs in the original JAVH and JAVC datasets, along with the size of the merged dataset. Additionally, it provides information on the average lengths of source codes and summaries.

Table 1. Summary of Dataset Statistics

Dataset	Filtered Pairs	Avg. Code Length	Avg. Summary Length
JAVH	7,749	1,761.24	22.90
JAVC	22,547	1,635.45	12.56
Merged	30,296	1,698.35	17.73

4.2 Evaluation Metrics

We evaluated the quality of generated summaries using three evaluation metrics: BLEU-4[17], METEOR[2] and ROUGE-L[12]. These metrics are widely used in machine translation and text summarization tasks to measure the similarity and quality between generated text and reference text [38]. For the task of code summarization generation, it can be viewed as translating source code into natural language, thus machine translation metrics can be effectively utilized to measure the quality of generated summaries. In our experiments, we adopt BLEU-4 as the reference metric. The specifics of each evaluation metric are outlined below:

BLEU is one of the fundamental evaluation metrics for code summarization generation. Its basic idea is to evaluate the degree of match between generated and reference summaries. It measures the similarity between the two by computing the precision of n-grams, where n ranges from 1 to 4. A higher BLEU score indicates a better alignment between generated and reference summaries.

The calculation is as follows:

$$BLEU - N = BP \times exp \sum_{n=1}^{N} \omega_n \log p_n \tag{8}$$

where p_n is the precision of n-grams in the generated summary that match the reference summary; ω_n is the weight for n-grams; and BP is the brevity penalty. In this experiment, we set N to 4.

METEOR is utilized to measure the match between the generated summary and the reference summary. It evaluates from two perspectives: lexical matching and word order reordering. For lexical matching, METEOR utilizes techniques such as word alignment and stemming to compute the lexical match between the generated and reference summaries. For word order reordering, METEOR considers adjustments and reordering of word sequences by computing the similarity between two sentences. A higher METEOR score indicates a higher similarity between the generated and reference summaries.

The formula for METEOR is as follows:

$$METEOR = \left(1 - \gamma \cdot frag^{\beta}\right) \cdot \frac{P \cdot R}{\alpha \cdot P + (1-\alpha) \cdot R} \tag{9}$$

where P and R are precision and recall respectively; β, γ are fixed parameters set to 0.9, 3.0, and 0.5 respectively.

ROUGE-L employs the longest common subsequence(LCS) to compute the matching degree between two sentences, indicating how much content in the generated summary is similar to the reference summary. It utilizes F-score, which represents the harmonic mean of precision and recall. A higher ROUGE-L score indicates a higher similarity between the machine-generated summary and the reference summary, indicating better summary quality.

The formula for ROUGE-L is as follows:

$$R_{lcs} = \frac{LCS(X,Y)}{m} \tag{10}$$

$$P_{lcs} = \frac{LCS(X,Y)}{n} \tag{11}$$

$$F_{lcs} = \frac{(1+\beta^2)R_{lcs}P_{lcs}}{R_{lcs}+\beta^2 P_{lcs}} \tag{12}$$

where $\beta = \frac{P_{lcs}}{R_{lcs}}$, F_{lcs} is the ROUGE-L score.

4.3 Comparative Experiments

To evaluate the effectiveness of SABCS, we conducted a comparative experiment with five models, including four pre-trained models (CodeBert, Gypsum, EACS, SASA) and a large language model (ChatGPT). During the comparative experiment, we set the hyperparameters of CodeBert, Gypsum, EACS, and SASA models according to the original papers and trained them on our dataset. Meanwhile, we tested the code summarization performance of ChatGPT using a question-answering approach.

CodeBert is a code comprehension approach based on the BERT model. It utilizes a pre-trained language model to acquire semantic representations of code, followed by fine-tuning on specific tasks. By engaging in pre-training and fine-tuning on a substantial volume of code data, CodeBert aims to enhance the model's performance on code comprehension tasks.

GypSum is amulti-encoder model based on the transformer framework. It consists of two encoders (G-Encoder and C-Encoder) and one decoder. The G-Encoder embeds the abstract syntax tree (AST) of the source code into a graph representation, while the C-Encoder converts the source code into a token sequence and performs word embedding. Both encoders provide mixed representations for the code, resulting in good performance in code summarization tasks.

EACS is an extraction and abstraction framework. The extraction model is used to extract important sentences containing key details from code fragments, and the abstraction model is used to generate code summaries based on the results of the extraction model. The combination of these two models makes the final output of the framework more in line with human writing standards.

ChatGPT is an AIGC model. It can generate explanations in the form of conversations based on the questions users ask. In the field of code summarization, users can submit the code they need to generate a summary to ChatGPT, and use it to automatically generate summaries for the code.

SASA introduces a structure-aware sparse attention mechanism intended to mitigate the memory and computational complexity escalation inherent in self-attention mechanisms in lengthy code comprehension tasks. Key components of this model include top-k sparse attention and structure-aware attention based on Abstract Syntax Trees (ASTs), thereby further amplifying the performance of lengthy code comprehension tasks.

5 Results

In this section, we provide a detailed analysis of our experimental results. We start by outlining the hyperparameters used in our study. Next, we examine the effectiveness of different components within SABCS through an ablation study. We then compare the performance of SABCS with other state-of-the-art models. Finally, we offer a qualitative analysis of the generated summaries to highlight the strengths and limitations of our model.

5.1 Experimental Hyperparameters

We will demonstrate the hyperparameters used in the SABCS experiment. We utilized the Pytorch deep learning framework to construct our model, and the training, validation, and testing sets were created using the filtered dataset introduced in Section 4.1. In Section 3.3, we mentioned Rule 1 and Rule 2, where the parameter values and experimental results are shown in Table 2. For this experiment, we set the learning rate to be 0.0001, batch size to be 16, maximum token count to be 1024, and iteration count to be 65. Additionally, we employed Adam optimizer for optimization with a gradient accumulation feature enabled at a step size of 8. During testing process, beam search was used for token generation with a beam size of 10. Other hyperparameters remained consistent with those of SASA model. The experimental values and results for learning rate can be found in Table 3 while those for batch size are presented in Table 4.

5.2 Ablation Study

The main idea of SABCS is to use two rules to allocate attention not only globally but also to longer code sequences, in order to preserve the structural information of long code as much as possible. To validate the effectiveness of each part of SABCS, we conducted ablation experiments by removing the rules from the model.

The results of these experiments are shown in Table 5. It can be observed that after removing Rule 1 and Rule 2, the model's scores on the test dataset generally decreased. Specifically, compared to SABCS, $w/oRule1$ showed a decrease in scores for BLEU-4, METEOR, and ROUGE-L by 0.59,1.66 and 1.95 respectively, demonstrating that Rule 1 helps allocate attention better to longer parts of the code.Similarly, $w/oRule2$ showed a decrease in scores by 0.53, 1.36 and 2 .19 respectively, indicating that Rule2 adjusts attention distribution based on logical distance between tokens.Furthermore, $w/oR1\ \&R2$ exhibited a decrease in scores by l .69, l .69 and 3 .99 respectively, further confirming the effectiveness of both Rulel and Rule2 in generating summaries for long code snippets.

Table 2. The values of Rule parameters and experimental results

		BLEU-4	METEOR	ROUGE-L
a	10	13.58	31.07	27.18
	15	13.72	31.31	27.45
	20	**13.84**	**31.52**	**27.68**
	25	13.66	31.21	27.33
	30	13.43	30.81	26.88
$w1$	1	13.84	31.52	27.68
	2	13.89	31.60	27.78
	3	**14.01**	**31.81**	**28.02**
	4	13.71	31.29	27.43
$w2$	1	14.01	31.81	28.02
	2	13.97	31.74	27.94
	3	**14.15**	**32.05**	**28.29**
	4	14.04	31.86	28.08
$w3$	1	14.15	32.05	28.29
	2	**14.25**	**32.77**	**29.11**
	3	14.07	31.91	28.13
	4	13.88	31.59	27.76
$w4$	1	**14.57**	**32.77**	**29.11**
	2	14.29	32.29	28.56
	3	14.17	32.08	28.32
	4	14.02	31.83	28.04

Table 3. The value of learning rate and experimental results

Learning Rate	BLEU-4
1e-3	13.66
5e-3	14.08
1e-4	**14.57**
5e-4	14.54
1e-5	14.32
5e-5	14.30

Table 4. The value of batch size and experimental results

Batch Size	BLEU-4
8	14.06
16	**14.57**
32	14.28

5.3 Comparative Experiments

Table 6 presents the overall performance of the model on the Java benchmark dataset. The best results for each metric are highlighted in bold. We observe that the SABCS model outperforms all comparison models on all evaluation metrics of the dataset. Compared to the second-best model, our scores have improved by 0.15, 0.29, and 0.88 in BLEU-4, METEOR, and ROUGE-L respectively. These results demonstrate the effectiveness of code-structured guided sparse attention mechanism in code summarization generation tasks.

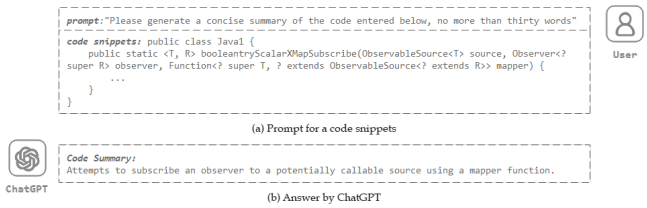
Table 5. Results of Ablation Study

Model	BLEU-4	ROUGE-L	METEOR
SABCS	14.57	32.77	29.11
$w/oRule1$	13.98	31.11	27.16
$w/oRule2$	14.04	31.41	26.92
$w/oR1\&R2$	12.88	31.08	25.12

Table 6. Results of Comparison experiments

Model	BLEU-4	ROUGE-L	METEOR
CodeBERT	13.09	30.23	26.22
GypSum	13.15	30.65	26.93
EACS	12.09	31.49	23.35
ChatGPT	14.42	32.48	28.23
SASA	13.10	31.08	25.12
SABCS	**14.57**	**32.77**	**29.11**

At the same time, we found that the EACS method performed relatively poorly on this dataset. We speculate that this may be due to the "extraction" part of EACS not being able to handle long code sequences effectively, resulting in incomplete extraction of key phrases. This subsequently affects the generation of code summaries in the "abstraction" part. CodeBert model only considers semantic information of the code and lacks structural information, leading to subpar performance. GypSum incorporates AST-based graph embedding and code sequence-based word embedding to preserve both structural and semantic information of the code. SASA utilizes sparse attention mechanism based on AST for code summarization but AST can only represent partial structural information of the code, hence their models perform slightly worse on this. ChatGPT based on pre-trained Transformer framework trained using internet text databases with a data volume of approximately 570GB, providing a solid foundation for its performance. In our comparative experiments with ChatGPT, we conducted model testing through interactive dialogues as shown in Figure 3. During testing, we observed that ChatGPT's output results contain numerous synonyms and are not concise enough, which resulted in lower scores across all three evaluation metrics.



(a) Prompt for a code snippets

(b) Answer by ChatGPT

Figure 3. the context of the interactive dialogues

```java
public class Java1 {
    public static <T, R> booleantryScalarXMapSubscribe(ObservableSource<T> source, Observer<?
super R> observer, Function<? super T, ? extends ObservableSource<? extends R>> mapper) {
        if (!(source instanceof Callable)) {
            returnfalse;
        }
        Tt= ((Callable<T>) source).call();
        if (t == null) {
            EmptyDisposable.complete(observer);
            returntrue;
        }
        ObservableSource<? extendsR> r = ObjectHelper.requireNonNull(mapper.apply(t), "The
mapper returned a null ObservableSource");
        if (r instanceof Callable) {
            Ru= ((Callable<R>) r).call();
            if (u == null) {
                EmptyDisposable.complete(observer);
                returntrue;
            }
            ScalarDisposable<R> sd = newScalarDisposable<>(observer, u);
            observer.onSubscribe(sd);
            sd.run();
        } else {
            r.subscribe(observer);
        }
        return true;
    }
}
```

(a) A code snippet from the test set

- **Gold Summary**: Tries to subscribe to a possibly Callable sources mapped Observable Source.
- **ChatGPT**: Attempts to subscribe an observer to a potentially callable source using a mapper function.
- **CodeBert**: Helper method to perform a scalar operation from a source and return map of the Observer.
- **EACS**: Performs a scalar x map operation on the source observable.
- **Gypsum**: Performs a copy of an Observor.
- **SASA**: Try to perform a clone of the source map.
- **SABCS**: Try to subscribe to a Observer of the source map.

(b) Summaries generated by different machines

Figure 4. An example from the test set

5.4 Example Analysis

Below is an example analysis of the comparative display of summaries generated by multiple models, provided for reference.

The figure 4 shows a Java example from the test set, where part (a) represents the code snippet to be predicted and part (b) displays the reference summary on the first line. The generated summaries based on ChatGPT, EACS, GypSum, SASA, and SABCS are shown in lines 2-6. From the figure, it can be observed that compared to the reference summary, GypSum's generated summary only partially covers the semantics of part three , i.e., Observor. Compared to GypSum, the abstract generated by EACS and CodeBert more fully covers the semantics of the third part, but it still doesn't work well. ChatGPT, SASA, and SABCS perform exceptionally well as they can adequately cover both parts one and three, and ChatGPT also explains the semantics of Part 2. However, in the abstracts generated by ChatGPT, there are many synonymous word substitutions e.g. , attempt to, try to, which affects its score in summarization quality evaluation.

In conclusion among all baseline models evaluated here, SABCS and ChatGPT perform well followed by SASA, CodeBert, GypSum, andEACS.

6 Conclusion And Future Work

In this paper, we introduce SABCS, a summarization generation model that effectively captures long-distance contextual information and deeply understands the semantics and syntax of code structures in long code snippets. By introducing global-local and Top-k attention mechanisms, as well as innovatively designing multi-dimensional structure-aware attention mechanisms based on AST and CFG, we limit the scope of attention, enabling the model to focus on code sequences and structural information, thereby improving the quality of long code summarization. Extensive experimental results demonstrate that SABCS outperforms existing models in performance, and through ablation experiments, the effectiveness of each part of SABCS is verified.

In the future, we will continue to explore new attention mechanisms to better capture fine-grained structural information in code snippets. Additionally, we will fine-tune the model for specific programming languages or domains to improve its applicability and performance.

References

[1] Frances E Allen. 1970. Control flow analysis. *ACM Sigplan Notices* 5, 7 (1970), 1–19.

[2] Satanjeev Banerjee and Alon Lavie. 2005. METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, Jade Goldstein, Alon Lavie, Chin-Yew Lin, and Clare Voss (Eds.). Association for Computational Linguistics, Ann Arbor, Michigan, 65–72. https://aclanthology.org/W05-0909

[3] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. *CoRR* abs/2004.05150 (2020). arXiv:2004.05150 https://arxiv.org/abs/2004.05150

[4] Srinadh Bhojanapalli, Ayan Chakrabarti, Andreas Veit, Michal Lukasik, Himanshu Jain, Frederick Liu, Yin-Wen Chang, and Sanjiv Kumar. 2021. Leveraging redundancy in attention with reuse transformers. *arXiv preprint arXiv:2110.06821* (2021).

[5] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, David Belanger, Lucy Colwell, et al. 2020. Masked language modeling for proteins via linearly scalable long-context transformers. *arXiv preprint arXiv:2006.03555* (2020).

[6] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. *CoRR* abs/2002.08155 (2020). arXiv:2002.08155 https://arxiv.org/abs/2002.08155

[7] Shuzheng Gao, Cuiyun Gao, Yulan He, Jichuan Zeng, Lunyiu Nie, Xin Xia, and Michael Lyu. 2023. Code structure–guided transformer for source code summarization. *ACM Transactions on Software Engineering and Methodology* 32, 1 (2023), 1–32.

[8] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2020. Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366* (2020).

[9] Qipeng Guo, Xipeng Qiu, Pengfei Liu, Yunfan Shao, Xiangyang Xue, and Zheng Zhang. 2022. Star-Transformer. arXiv:1902.09113 [cs.CL]

[10] Sonia Haiduc, Jairo Aponte, and Andrian Marcus. 2010. Supporting program comprehension with source code summarization. In *2010 ACM/IEEE 32nd International Conference on Software Engineering*, Vol. 2. 223–226. https://doi.org/10.1145/1810295.1810335

[11] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. *CoRR* abs/1909.09436 (2019). arXiv:1909.09436 http://arxiv.org/abs/1909.09436

[12] Chin-Yew Lin. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Annual Meeting of the Association for Computational Linguistics*. https://api.semanticscholar.org/CorpusID:964287

[13] Shangqing Liu, Yu Chen, Xiaofei Xie, Jingkai Siow, and Yang Liu. 2020. Retrieval-augmented generation for code summarization via hybrid gnn. *arXiv preprint arXiv:2006.05405* (2020).

[14] Tingting Liu, Chengyu Wang, Cen Chen, Ming Gao, and Aoying Zhou. 2022. Understanding Long Programming Languages with Structure-Aware Sparse Attention. arXiv:2205.13730 [cs.CL]

[15] John McCarthy. 1960. Recursive functions of symbolic expressions and their computation by machine, Part I. *Commun. ACM* 3, 4 (apr 1960), 184–195. https://doi.org/10.1145/367177.367199

[16] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781* (2013).

[17] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, Pierre Isabelle, Eugene Charniak, and Dekang Lin (Eds.). Association for Computational Linguistics, Philadelphia, Pennsylvania, USA, 311–318. https://doi.org/10.3115/1073083.1073135

[18] Ensheng Shi, Yanlin Wang, Lun Du, Hongyu Zhang, Shi Han, Dongmei Zhang, and Hongbin Sun. 2021. Cast: Enhancing code summarization with hierarchical splitting and reconstruction of abstract syntax trees. *arXiv preprint arXiv:2108.12987* (2021).

[19] Weisong Sun, Chunrong Fang, Yuchen Chen, Quanjun Zhang, Guanhong Tao, Yudu You, Tingxu Han, Yifei Ge, Yuling Hu, Bin Luo, et al. 2024. An extractive-and-abstractive framework for source code summarization. *ACM Transactions on Software Engineering and Methodology* 33, 3 (2024), 1–39.

[20] Yao Wan, Zhou Zhao, Min Yang, Guandong Xu, Haochao Ying, Jian Wu, and Philip S. Yu. 2018. Improving Automatic Source Code Summarization via Deep Reinforcement Learning. *CoRR* abs/1811.07234 (2018). arXiv:1811.07234 http://arxiv.org/abs/1811.07234

[21] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768* (2020).

[22] Yu Wang, Yu Dong, Xuesong Lu, and Aoying Zhou. 2022. Gypsum: learning hybrid representations for code summarization. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*. 12–23.

[23] Yongqiang Wang, Yangyang Shi, Frank Zhang, Chunyang Wu, Julian Chan, Ching-Feng Yeh, and Alex Xiao. 2021. Transformer in action: a comparative study of transformer-based acoustic models for large scale speech recognition applications. In *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 6778–6782.

[24] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big Bird: Transformers for Longer Sequences. *CoRR* abs/2007.14062 (2020). arXiv:2007.14062 https://arxiv.org/abs/2007.14062

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009